several ways to achieve this, but in this section I introduce the `OptionsBuilder<T>` as one possible approach and highlight some of the other features it enables.

In chapter 10 we discussed the ASP.NET Core configuration system in depth. You saw how an `IConfiguration` object is built from multiple layers, where subsequent layers can add to or replace configuration values from previous layers. Each layer is added by a configuration provider, which reads values from a file, from environment variables, from User Secrets, or from any number of possible locations.

A common and encouraged practice is to bind your configuration object to strongly typed `IOptions<T>` objects, as you saw in chapter 10. Typically, you configure this binding in Program.cs by calling `builder.Services.Configure<T>()` and providing an `IConfiguration` object or a configuration section to bind.

For example, to bind a strongly typed object called `CurrencyOptions` to the `"Currencies"` section of an `IConfiguration` object, you could use the following:

```
builder.services.Configure<CurrencyOptions>(
    Configuration.GetSection("Currencies"));
```

TIP You can see an example of the `CurrencyOptions` type and the associated `"Currencies"` section of appsetttings.json in the source code for this chapter.

This sets the properties of the `CurrencyOptions` object, based on the values in the `"Currencies"` section of your `IConfiguration` object. Simple binding like this is common, but sometimes you might not